Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence and Machine Learning

Report on Mini Project

Machine Learning -I (DJS23DPC402L)

AY: 2025-26

TITLE OF THE PROJECT
Driving Behaviour Analysis Using Machine Learning: A Multi-Class Classification of Aggressive, Normal, and Slow Driving from Smartphone Sensor Data

---

Manas Shah A1-2 A132 60017250013
Yash Poojari A1-2 A140 60017250010
Tirrth Mistry A1-2 A138 60017250015
Deep Shah A1-2 A054 60017240169
Arth Patel A1-2 A039 60017240164

Guided By:
Dr. Kriti Srivastava

```
print("DATASET LOADED")
print(f"\nDataset Shape: {df.shape}")
print(f"Number of Records: {df.shape[0]}")
print(f"Number of Features: {df.shape[1]}")
print(f"\nColumn Names:\n{df.columns.tolist()}")
print(f"\nData Types:\n{df.dtypes}")
print(f"\nFirst 5 Rows:\n{df.head()}")
```

```
DATASET LOADED

Dataset Shape: (3644, 8)
Number of Records: 3644
Number of Features: 8

Column Names:
['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY', 'GyroZ', 'Class',
'Timestamp']

Data Types:
AccX         float64
AccY         float64
AccZ         float64
GyroX        float64
GyroY        float64
GyroZ        float64
Class         object
Timestamp      int64
dtype: object

First 5 Rows:
      AccX      AccY      AccZ     GyroX     GyroY     GyroZ   Class
\
0  0.000000  0.000000  0.000000  0.059407 -0.174707  0.101938  NORMAL

1 -1.624864 -1.082492 -0.204183 -0.028558  0.051313  0.135536  NORMAL

2 -0.594660 -0.122410  0.220502 -0.019395 -0.029322  0.087888  NORMAL

3  0.738478 -0.228456  0.667732  0.069791 -0.029932  0.054902  NORMAL

4  0.101741  0.777568 -0.066730  0.030696 -0.003665  0.054902  NORMAL


   Timestamp
0    3581629
1    3581630
2    3581630
3    3581631
4    3581631
```

1. DATA QUALITY & PREPROCESSING

```
print("DATA QUALITY CHECK")

print(f"\nMissing Values:\n{df.isnull().sum()}")
print(f"\nTotal Missing Percentage: {(df.isnull().sum().sum() /
(df.shape[0]*df.shape[1])) * 100:.2f}%")

print(f"\nDuplicate Rows: {df.duplicated().sum()}")

print(f"\nData Types Info:")
print(df.dtypes)

print(f"\nTarget Variable Distribution (Class):")
print(df['Class'].value_counts())
print(f"\nTarget Distribution Percentage:")
print(df['Class'].value_counts(normalize=True) * 100)
```

```
DATA QUALITY CHECK

Missing Values:
AccX         0
AccY         0
AccZ         0
GyroX        0
GyroY        0
GyroZ        0
Class        0
Timestamp    0
dtype: int64

Total Missing Percentage: 0.00%

Duplicate Rows: 0

Data Types Info:
AccX         float64
AccY         float64
AccZ         float64
GyroX        float64
GyroY        float64
GyroZ        float64
Class         object
Timestamp      int64
dtype: object

Target Variable Distribution (Class):
Class
SLOW          1331
NORMAL        1200
AGGRESSIVE    1113
Name: count, dtype: int64
```

```
                      50%           75%           max
AccX        3.931045e-03  5.959871e-01  4.985548e+00
AccY       -8.083267e-02  4.524008e-01  4.245151e+00
AccZ        2.261639e-03  5.561569e-01  5.171739e+00
GyroX       1.985312e-03  3.191771e-02  8.492554e-01
GyroY      -1.832596e-03  5.131268e-02  1.679879e+00
GyroZ       2.977968e-03  4.085161e-02  1.190500e+00
Timestamp   3.582702e+06  3.583270e+06  3.583791e+06


OUTLIER DETECTION (IQR Method):
AccX: 112 outliers (3.07%)
AccY: 128 outliers (3.51%)
AccZ: 125 outliers (3.43%)
GyroX: 193 outliers (5.30%)
GyroY: 230 outliers (6.31%)
GyroZ: 389 outliers (10.68%)

Total Outliers Found: 1177
```

1. EXPLORATORY DATA ANALYSIS (EDA)

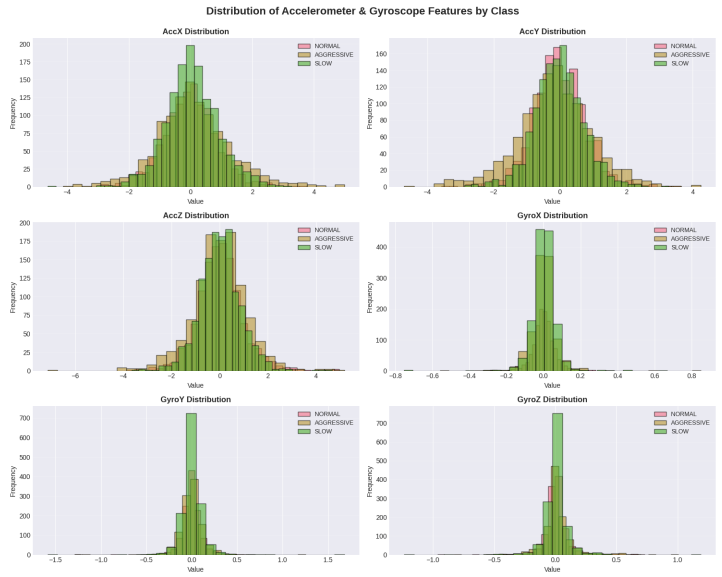
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

class_counts = df['Class'].value_counts()
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
axes[0].bar(class_counts.index, class_counts.values, color=colors,
edgecolor='black', linewidth=1.5)
axes[0].set_title('Driving Behavior Class Distribution', fontsize=14,
fontweight='bold')
axes[0].set_xlabel('Behavior Class', fontsize=12)
axes[0].set_ylabel('Count', fontsize=12)
axes[0].grid(axis='y', alpha=0.3)

axes[1].pie(class_counts.values, labels=class_counts.index,
autopct='%1.1f%%',
            colors=colors, startangle=90, textprops={'fontsize': 11,
'fontweight': 'bold'})
axes[1].set_title('Class Distribution Percentage', fontsize=14,
fontweight='bold')

plt.tight_layout()
plt.show()

print("✓ Class distribution looks balanced (no severe class
imbalance)")
```




```
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Box Plots: Feature Ranges by Driving Behavior Class',
             fontsize=16, fontweight='bold')

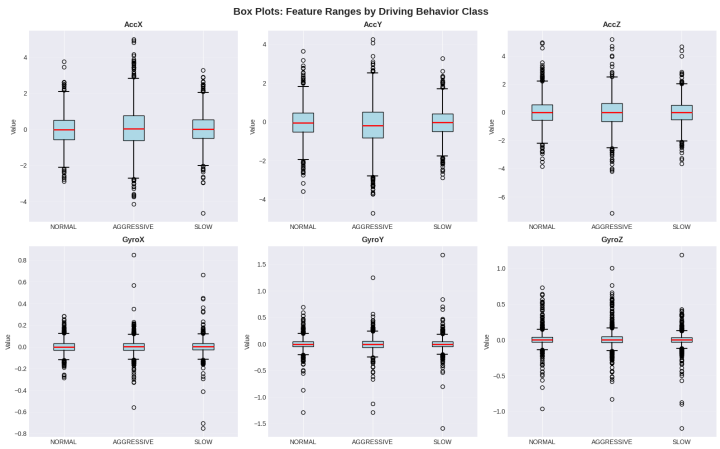
for idx, feature in enumerate(feature_list):
    ax = axes[idx // 3, idx % 3]

    data_to_plot = [df[df['Class'] == c][feature].values for c in
classes]
    bp = ax.boxplot(data_to_plot, labels=classes, patch_artist=True,
                    medianprops=dict(color='red', linewidth=2),
                    boxprops=dict(facecolor='lightblue',
edgecolor='black'),
                    whiskerprops=dict(color='black', linewidth=1.5),
                    capprops=dict(color='black', linewidth=1.5))

    ax.set_title(f'{feature}', fontsize=12, fontweight='bold')
    ax.set_ylabel('Value', fontsize=10)
    ax.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
plt.show()

print("✓ Box plots show clear separation between classes for some features")
```




```
✓ Box plots show clear separation between classes for some features
```

```
correlation_matrix = df[feature_list].corr()

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f',
cmap='coolwarm',
            center=0, square=True, linewidths=1, cbar_kws={"shrink":
0.8},
            ax=ax)
ax.set_title('Correlation Matrix of Features', fontsize=14,
fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

print("\nKey Correlations:")
corr_pairs = []
for i in range(len(correlation_matrix.columns)):
    for j in range(i+1, len(correlation_matrix.columns)):
        corr_pairs.append((
```

```
print(f"\nTOTAL FEATURES FOR MODELING: {len(original_features) +
len(engineered_features)}")
print(f"\nDataset shape after feature engineering:
{df_features.shape}")
```

```
FEATURE ENGINEERING SUMMARY



Original Features: 6
  ['AccX', 'AccY', 'AccZ', 'GyroX', 'GyroY', 'GyroZ']

Engineered Features: 32

  Jerk Features (17): ['JerkX', 'JerkY', 'JerkZ', 'JerkMagnitude',
'JerkX_rolling_mean', 'JerkY_rolling_mean', 'JerkZ_rolling_mean',
'JerkX_rolling_max', 'JerkY_rolling_max', 'JerkZ_rolling_max',
'JerkX_var', 'JerkY_var', 'JerkZ_var', 'AbsJerkX', 'AbsJerkY',
'AbsJerkZ', 'Jerk_Energy']
  Magnitude Features (3): ['AccMagnitude', 'GyroMagnitude',
'JerkMagnitude']
  Rolling Window Features (12): ['AccX_rolling_mean',
'AccY_rolling_mean', 'AccZ_rolling_mean', 'AccX_rolling_std',
'AccY_rolling_std', 'AccZ_rolling_std', 'JerkX_rolling_mean',
'JerkY_rolling_mean', 'JerkZ_rolling_mean', 'JerkX_rolling_max',
'JerkY_rolling_max', 'JerkZ_rolling_max']
  Other Features (7): ['AccX_var', 'AccY_var', 'AccZ_var', 'AbsAccX',
'AbsAccY', 'AbsAccZ', 'Acc_Energy']

TOTAL FEATURES FOR MODELING: 38

Dataset shape after feature engineering: (3644, 40)
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('Jerk Distribution by Driving Behavior Class',
fontsize=14, fontweight='bold')

jerk_cols = ['JerkX', 'JerkY', 'JerkZ']
for idx, col in enumerate(jerk_cols):
    for class_label in sorted(df_features['Class'].unique()):
        data = df_features[df_features['Class'] == class_label][col]
        axes[idx].hist(data, alpha=0.6, label=class_label, bins=40,
edgecolor='black')

    axes[idx].set_title(f'{col} Distribution', fontweight='bold')
    axes[idx].set_xlabel('Jerk Value')
    axes[idx].set_ylabel('Frequency')
    axes[idx].legend()
    axes[idx].grid(axis='y', alpha=0.3)

plt.tight_layout()
```

```
print(f"\nTraining Set Class Distribution:")
for u, c in zip(*np.unique(y_train, return_counts=True)):
    class_name = label_encoder.inverse_transform([u])[0]
    print(f"  {class_name}: {c} samples")
print(f"\nTest Set Class Distribution:")
for u, c in zip(*np.unique(y_test, return_counts=True)):
    class_name = label_encoder.inverse_transform([u])[0]
    print(f"  {class_name}: {c} samples")


Train-Test Split (80-20):
Training Set Size: 2915 samples (79.99%)
Test Set Size: 729 samples (20.01%)

Training Set Class Distribution:
  AGGRESSIVE: 890 samples
  NORMAL: 960 samples
  SLOW: 1065 samples

Test Set Class Distribution:
  AGGRESSIVE: 223 samples
  NORMAL: 240 samples
  SLOW: 266 samples

print("\nFeature Scaling:")
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("✓ Features scaled using StandardScaler")
print(f"\nScaled Training Data Statistics:")
print(f"  Mean: {X_train_scaled.mean():.6f}")
print(f"  Std Dev: {X_train_scaled.std():.6f}")
print(f"  Min: {X_train_scaled.min():.6f}")
print(f"  Max: {X_train_scaled.max():.6f}")


Feature Scaling:
✓ Features scaled using StandardScaler

Scaled Training Data Statistics:
  Mean: -0.000000
  Std Dev: 1.000000
  Min: -16.793035
  Max: 23.610539
```

6. MODEL TRAINING & EVALUATION

```
models_dict = {}
results_dict = {}
```

```
print("TRAINING MULTIPLE SCIKIT-LEARN MODELS")

print("\n1. LOGISTIC REGRESSION\n")

lr_model = LogisticRegression(max_iter=1000, random_state=42,
multi_class='multinomial')
lr_model.fit(X_train_scaled, y_train)
y_pred_lr = lr_model.predict(X_test_scaled)
y_pred_proba_lr = lr_model.predict_proba(X_test_scaled)

models_dict['Logistic Regression'] = lr_model
results_dict['Logistic Regression'] = {
    'predictions': y_pred_lr,
    'probabilities': y_pred_proba_lr,
    'accuracy': accuracy_score(y_test, y_pred_lr),
    'precision': precision_score(y_test, y_pred_lr,
average='weighted'),
    'recall': recall_score(y_test, y_pred_lr, average='weighted'),
    'f1': f1_score(y_test, y_pred_lr, average='weighted')
}

print(f"Accuracy: {results_dict['Logistic Regression']
['accuracy']:.4f}")
print(f"Precision (Weighted): {results_dict['Logistic Regression']
['precision']:.4f}")
print(f"Recall (Weighted): {results_dict['Logistic Regression']
['recall']:.4f}")
print(f"F1-Score (Weighted): {results_dict['Logistic Regression']
['f1']:.4f}")
```

```
TRAINING MULTIPLE SCIKIT-LEARN MODELS

1. LOGISTIC REGRESSION

Accuracy: 0.5926
Precision (Weighted): 0.5940
Recall (Weighted): 0.5926
F1-Score (Weighted): 0.5893

print("\n2. RANDOM FOREST CLASSIFIER\n \n")

rf_model = RandomForestClassifier(n_estimators=100, max_depth=20,
                                  random_state=42, n_jobs=-1,
min_samples_split=5)
rf_model.fit(X_train_scaled, y_train)
y_pred_rf = rf_model.predict(X_test_scaled)
y_pred_proba_rf = rf_model.predict_proba(X_test_scaled)

models_dict['Random Forest'] = rf_model
results_dict['Random Forest'] = {
```

```
Recall (Weighted): 0.6063
F1-Score (Weighted): 0.6048
```

1. COMPREHENSIVE MODEL COMPARISON & EVALUATION

```
print("MODEL PERFORMANCE COMPARISON\n \n")


comparison_df = pd.DataFrame(results_dict).T[['accuracy', 'precision',
'recall', 'f1']].round(4)
comparison_df = comparison_df.sort_values('f1', ascending=False)

print(f"\nAll Models Performance Metrics:")
print(comparison_df)

print(f"\nBest Model by F1-Score: {comparison_df['f1'].idxmax()}
({comparison_df['f1'].max():.4f})")
print(f"Best Model by Accuracy: {comparison_df['accuracy'].idxmax()}
({comparison_df['accuracy'].max():.4f})")
```

```
MODEL PERFORMANCE COMPARISON



All Models Performance Metrics:
                    accuracy precision    recall        f1
Gradient Boosting     0.99177  0.991806   0.99177  0.991749
Random Forest        0.958848  0.959403  0.958848  0.958942
Decision Tree        0.894376  0.895771  0.894376   0.89443
SVM                  0.625514  0.624466  0.625514   0.62334
KNN                  0.603567  0.616056  0.603567  0.607747
AdaBoost              0.60631  0.609436   0.60631  0.604815
Logistic Regression  0.592593  0.594034  0.592593  0.589333
Naive Bayes          0.491084  0.479123  0.491084  0.459544

Best Model by F1-Score: Gradient Boosting (0.9917)
Best Model by Accuracy: Gradient Boosting (0.9918)
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Model Performance Comparison', fontsize=16,
fontweight='bold')

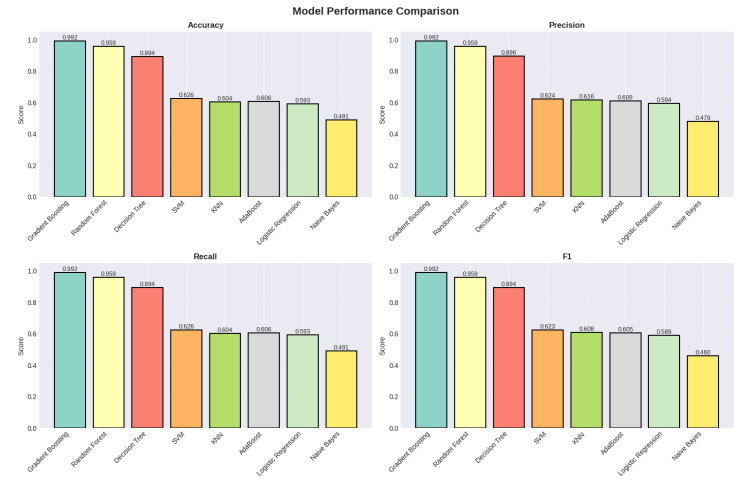
metrics = ['accuracy', 'precision', 'recall', 'f1']
colors_models = plt.cm.Set3(np.linspace(0, 1, len(comparison_df)))

for idx, metric in enumerate(metrics):
    ax = axes[idx // 2, idx % 2]
    values = comparison_df[metric].values
    bars = ax.bar(range(len(comparison_df)), values,
color=colors_models, edgecolor='black', linewidth=1.5)
```

```
    ax.set_xticks(range(len(comparison_df)))
    ax.set_xticklabels(comparison_df.index, rotation=45, ha='right')
    ax.set_title(f'{metric.capitalize()}', fontweight='bold')
    ax.set_ylabel('Score', fontsize=11)
    ax.set_ylim([0, 1.05])
    ax.grid(axis='y', alpha=0.3)

    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.3f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()
```




```
best_model_name = comparison_df['f1'].idxmax()
best_model = models_dict[best_model_name]
best_predictions = results_dict[best_model_name]['predictions']

print("\n \n")
print(f"DETAILED EVALUATION - BEST MODEL: {best_model_name}")
print("\n \n")

print(f"\nClassification Report:\n")
```

```
print(classification_report(y_test, best_predictions,
                            target_names=label_encoder.classes_,
                            digits=4))

print(f"\nConfusion Matrix:")
cm = confusion_matrix(y_test, best_predictions)
print(cm)
print(f"\nConfusion Matrix (Decoded):")
cm_df = pd.DataFrame(cm, index=label_encoder.classes_,
columns=label_encoder.classes_)
print(cm_df)
```

```
DETAILED EVALUATION - BEST MODEL: Gradient Boosting



Classification Report:

              precision    recall  f1-score   support

  AGGRESSIVE     0.9954    0.9776    0.9864       223
      NORMAL     0.9877    1.0000    0.9938       240
        SLOW     0.9925    0.9962    0.9944       266

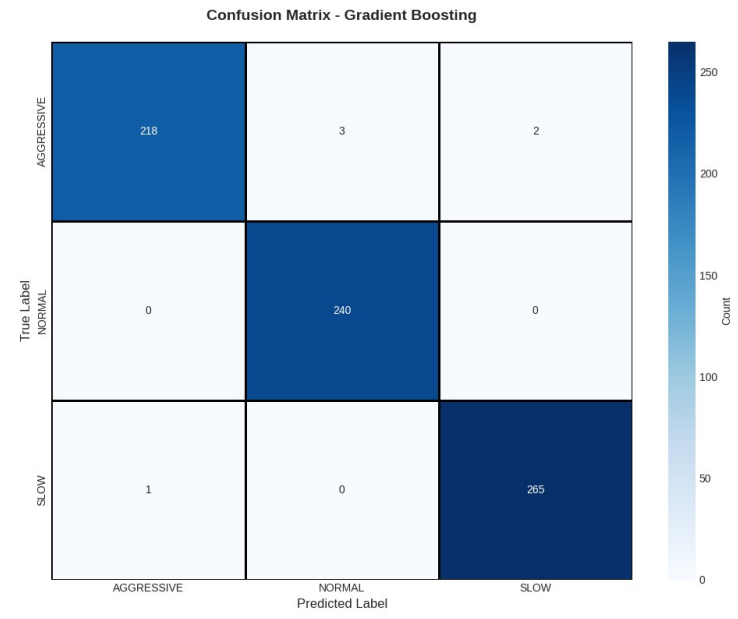
    accuracy                         0.9918       729
   macro avg     0.9919    0.9913    0.9915       729
weighted avg     0.9918    0.9918    0.9917       729


Confusion Matrix:
[[218   3   2]
 [  0 240   0]
 [  1   0 265]]

Confusion Matrix (Decoded):
            AGGRESSIVE  NORMAL  SLOW
AGGRESSIVE         218       3     2
NORMAL               0     240     0
SLOW                 1       0   265
```

```
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_,
            cbar_kws={'label': 'Count'},
            linewidths=2, linecolor='black', ax=ax)
ax.set_title(f'Confusion Matrix - {best_model_name}', fontsize=14,
```

```
fontweight='bold', pad=20)
ax.set_ylabel('True Label', fontsize=12)
ax.set_xlabel('Predicted Label', fontsize=12)
plt.tight_layout()
plt.show()
```




```
print("\n \n")
print(f"CROSS-VALIDATION ANALYSIS - {best_model_name}")
print("\n \n")


kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(best_model, X_train_scaled, y_train,
cv=kfold, scoring='f1_weighted')

print(f"\n5-Fold Cross-Validation F1-Scores:")
for i, score in enumerate(cv_scores, 1):
```

```
    print(f"  Fold {i}: {score:.4f}")

print(f"\nMean CV Score: {cv_scores.mean():.4f}")
print(f"Std Dev: {cv_scores.std():.4f}")
print(f"Confidence Interval (95%): [{cv_scores.mean() -
1.96*cv_scores.std():.4f}, {cv_scores.mean() +
1.96*cv_scores.std():.4f}]")



CROSS-VALIDATION ANALYSIS - Gradient Boosting



5-Fold Cross-Validation F1-Scores:
  Fold 1: 0.9811
  Fold 2: 0.9777
  Fold 3: 0.9845
  Fold 4: 0.9949
  Fold 5: 0.9914

Mean CV Score: 0.9859
Std Dev: 0.0064
Confidence Interval (95%): [0.9735, 0.9984]

if best_model_name in ['Random Forest', 'Gradient Boosting', 'Decision
Tree', 'AdaBoost']:
    print("\n \n")
    print(f"FEATURE IMPORTANCE - {best_model_name}")


    feature_imp = pd.DataFrame({
        'Feature': feature_cols,
        'Importance': best_model.feature_importances_
    }).sort_values('Importance', ascending=False)

    print(f"\nTop 15 Most Important Features:")
    print(feature_imp.head(15).to_string(index=False))

    fig, ax = plt.subplots(figsize=(12, 8))
    top_features = feature_imp.head(15)
    bars = ax.barh(range(len(top_features)),
top_features['Importance'].values,
                   color=plt.cm.viridis(np.linspace(0, 1,
len(top_features))),
                   edgecolor='black', linewidth=1.5)
    ax.set_yticks(range(len(top_features)))
    ax.set_yticklabels(top_features['Feature'].values)
    ax.set_xlabel('Importance Score', fontsize=12, fontweight='bold')
```

```
    ax.set_title(f'Top 15 Feature Importance - {best_model_name}',
fontsize=14, fontweight='bold')
    ax.grid(axis='x', alpha=0.3)

    for i, bar in enumerate(bars):
        width = bar.get_width()
        ax.text(width, bar.get_y() + bar.get_height()/2.,
                f'{width:.4f}', ha='left', va='center', fontsize=10)

    plt.tight_layout()
    plt.show()



FEATURE IMPORTANCE - Gradient Boosting

Top 15 Most Important Features:
          Feature  Importance
  AccY_rolling_std    0.255418
 JerkY_rolling_max    0.156008
 JerkZ_rolling_max    0.132036
 JerkX_rolling_max    0.130890
  AccX_rolling_std    0.120834
 AccZ_rolling_mean    0.055665
  AccZ_rolling_std    0.042826
 AccY_rolling_mean    0.037653
 AccX_rolling_mean    0.033461
             GyroZ    0.006771
     GyroMagnitude    0.005395
              AccX    0.003184
JerkY_rolling_mean    0.002385
             GyroX    0.001914
JerkX_rolling_mean    0.001250
```

```
Grid Search Configuration:
Parameter combinations to test: 18
Fitting 3 folds for each of 18 candidates, totalling 54 fits

Best Parameters: {'learning_rate': 0.15, 'max_depth': 7,
'n_estimators': 150}
Best Cross-Validation Score: 0.9876

Optimized Model Test Performance:
  Accuracy: 0.9959
  F1-Score: 0.9959

Improvement:
  Accuracy: 0.0041
  F1-Score: 0.0041
```

1. PREDICTION ON NEW DATA

```
print("\n")
print("EXAMPLE: PREDICTION ON NEW DATA")
print("\n \n")


sample_indices = np.random.choice(len(X_test), 5, replace=False)
X_sample = X_test.iloc[sample_indices]
y_sample = y_test[sample_indices]

print("\nSample Data for Prediction:")
for idx, sample_idx in enumerate(sample_indices, 1):
    true_label = label_encoder.inverse_transform([y_sample[idx-1]])[0]
    print(f"\nSample {idx}:")
    print(f"  Feature Values (first 6): {X_sample.iloc[idx-
1, :6].values}")
    print(f"  True Class: {true_label}")

    X_sample_scaled = scaler.transform(X_sample.iloc[[idx-1]])
    prediction = best_model.predict(X_sample_scaled)[0]
    probabilities = best_model.predict_proba(X_sample_scaled)[0]
    predicted_label = label_encoder.inverse_transform([prediction])[0]

    print(f"  Predicted Class: {predicted_label}")
    print(f"  Prediction Confidence:")
    for class_idx, class_name in enumerate(label_encoder.classes_):
        print(f"    - {class_name}: {probabilities[class_idx]:.4f}")

    match = "✓" if true_label == predicted_label else "✗"
    print(f"  Match: {match}")
```

```
cv_test_gap = cv_scores.mean() - test_f1

print(f"Train - Test Gap : {train_test_gap:.4f}")
print(f"CV - Test Gap    : {cv_test_gap:.4f}")

if train_test_gap > 0.05:
    print(" Possible Overfitting (train >> test)")
elif test_f1 > train_f1:
    print(" Good Generalization (test >= train)")
else:
    print(" No significant overfitting detected")

============================================================
OVERFITTING CHECK
============================================================

 TRAIN PERFORMANCE
Accuracy : 1.0000
F1 Score : 1.0000

 TEST PERFORMANCE
Accuracy : 0.9959
F1 Score : 0.9959

 CROSS-VALIDATION (5-Fold)
Scores   : [0.9914304  0.99314674 0.99143219 0.9931407  0.99485033]
Mean F1  : 0.9928

 ANALYSIS
----------------------------------------
Train - Test Gap : 0.0041
CV - Test Gap    : -0.0031
 No significant overfitting detected
```